Studio Nx

Backend(FastAPI/Python) 07 — get_current_user_optional()

2026-10-06

Backend(FastAPI/Python) 07 — `get_current_user_optional()`

항목	내용
파일	`studionx/backend/app/api/deps.py`
작업	P1-03 백엔드 공통 — 로그인이 선택인 API · PRD 01 FR-112
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	06 `get_current_user()` · 08 `User` 모델

1. 하는 일

로그인한 회원이면 그 회원을, 아니면 `None`(손님)을 라우터에 넘긴다. **401 오류를 내지 않는다.** 06 과 검사는 같고, 실패했을 때의 처리만 다르다.

	06 `get_current_user()`	07 이 함수
로그인한 `ACTIVE` 회원	회원	회원
헤더 없음 · 토큰 틀림 · 회원 없음 · 정지 · 탈퇴	401 오류 (라우터 실행 안 됨)	`None` (라우터는 손님으로 실행)
라우터에 적는 이름	`user: CurrentUser`	`user: OptionalUser`

2. 언제 쓰나

지금 쓰는 곳은 **문의 등록** `POST /api/v1/inquiries` 하나다 (home.html 아래쪽 Contact 폼).

보내는 사람	이 함수의 결과	저장되는 문의 (`inquiries` 테이블)
로그인 안 한 방문자	`None`	`user_id = NULL` — 폼에 적은 이메일로만 답장
로그인한 회원 7	7번 회원	`user_id = 7` — 관리자가 어느 회원의 문의인지 알 수 있음
토큰이 만료된 회원	`None`	`user_id = NULL` — 문의는 그대로 접수됨

```
def create_inquiry(data: InquiryCreateRequest, request: Request, user: OptionalUser, db: Db):
    ...   # P1-12: user 가 있으면 inquiry.user_id = user.user_id
```

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import Depends
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import decode_access_token
```

```
from app.db.session import get_db
from app.models import User
```

`deps.py` 맨 위의 import 이다 (06 과 같음). 이 함수는 **`AppError`** 만 직접 쓴다. 나머지 검사는 06 `get_current_user()` 를 불러서 한다.

함수

```
def get_current_user_optional(
    credentials: Bearer,
    db: Db,
) -> User | None:
    if credentials is None:
        return None
    try:
        return get_current_user(credentials, db)
    except AppError:
        return None


OptionalUser = Annotated[User | None, Depends(get_current_user_optional)]
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀

식	뜻
`-> User | None`	회원 또는 `None` 을 돌려준다는 표시
`get_current_user(credentials, db)`	06 을 **보통 함수로 직접** 부름. `Depends` 를 거치지 않으므로 이미 받은 `credentials` · `db` 를 그대로 넘김
`try: … except AppError: return None`	06 이 던진 401(`AppError`)을 잡아 `None` 으로 바꿈. 05 의 토큰 오류도 `AppError` 라서 함께 잡힘
`except AppError` (모든 오류가 아님)	DB 가 끊긴 오류 같은 것은 잡지 않음 → 04 가 500 으로 처리

4. 흐름

1. 헤더가 없으면 `None`. DB 에 묻지 않는다.
2. 있으면 06 과 같은 검사를 한다.
3. 통과하면 회원, 401 이면 `None`.

5. 왜 이렇게

선택	이유
실패해도 401 을 내지 않음	문의는 손님도 보낼 수 있다. 만료된 토큰 때문에 문의가 거절되면 안 된다
검사를 06 에 맡김	검사 규칙(토큰 · 회원 없음 · 정지 · 탈퇴)이 한 곳에만 있다. 06 을 고치면 07 도 같이 바뀐다

선택	이유
`AppError` 만 잡음	DB 오류까지 `None` 으로 바꾸면 서버 고장이 숨는다. 고장은 04 가 500 으로 알리고 Sentry 에 기록한다
정지 · 탈퇴 회원도 `None`	그 회원 이름으로 연결하지 않고 손님 문의로 받는다

P1-12 에서 알아 둘 것: Access Token 은 메모리에만 있어서 home.html 을 새로 열면 비어 있다 (JS 01). 로그인한 회원의 문의를 회원과 연결하려면, `home.js` 가 보내기 전에 `isLoggedIn()` (JS 06)으로 토큰을 먼저 받아 두어야 한다.

6. 시험 (`tests/test_deps.py` 에 6개 추가 · 모두 13개)

06 과 같은 가짜 세션(`FakeDb`)을 쓰고, 시험용 앱에 `GET /maybe` (`user: OptionalUser` → `{"user_id": 7 또는 null}`)를 만들었다.

경우	결과
정상 토큰 + `ACTIVE` 회원 7	200 `{"user_id": 7}`
헤더 없음	200 `{"user_id": null}` · DB 에 묻지 않음
틀린 토큰	200 `{"user_id": null}`
토큰 회원 99 가 DB 에 없음	200 `{"user_id": null}`
`DELETED` 회원	200 `{"user_id": null}`
`Authorization: Basic …`	200 `{"user_id": null}`

전체 시험 48개 통과, `ruff` 검사 통과.